

Contents

2. Dataset Description	2
2.1 Dataset Components	3
2.2 Users.csv	3
2.3 Posts.csv	4
2.4 Comments.csv	4
2.5 Tags.csv	5
2.6 Dataset Size	5
3. Dataset Preparation	6
3.1 HDFS Input Files	6
4. Hadoop and HDFS Setup	6
4.1 Hadoop Components	7
4.2 Hadoop Commands	7
5. Summary of Data Analysis	7
6. MapReduce Analysis	8
6.1 Users with Website	8
6.2 Distinct User Locations	10
6.3 Top 10 Users by Reputation	11
6.4 Top 10 Post Scores	13
6.5 Count PostTypeId	15
7. Pig Analysis	17
7.1 High Reputation Users	17
7.2 Users With Website	18
7.3 Distinct Locations	18
7.4 Top 10 Users by Reputation	19
7.5 Posts With Score Greater Than 10	20
7.6 Top 10 Posts by Score	21
7.7 Top 10 Viewed Posts	22
7.9 Posts Grouped by Post Type	23
7.10 Join Users With Posts	24
7.11 Join Posts With Comments	25
7.12 Top 10 Tags	26
7.13 Top 10 Users by Number of Posts	27
7.14 Pig Output Structure	28
8. Discussion	28
9. Conclusion	30

1. Problem Statement

The objective of this project is to process and analyze a large Stack Exchange Database Administrator (DBA) dataset using Hadoop Distributed File System (HDFS), Java MapReduce, and Apache Pig.

The Stack Exchange DBA dataset contains information about users, questions, answers, comments, and technical tags. Because the dataset consists of multiple large files and contains relationships between different entities, it provides a suitable environment for demonstrating big-data storage, processing, aggregation, ranking, and join operations.

The main purpose of this project is to demonstrate how Hadoop-based technologies can be used to process large datasets efficiently instead of relying only on traditional single-machine data-processing techniques.

The project focuses on the following analytical questions:

- How many users have a personal website?
- What are the distinct locations of users?
- Which users have the highest reputation?
- What are the highest post scores?
- Which posts have the highest scores?
- Which posts have the highest number of views?
- Which posts have many comments?
- How are posts distributed by post type?
- Which users have created the most posts?
- Which users have made the most comments?
- Which tags are most popular?
- How are users, posts, and comments related?

To answer these questions, the project uses both low-level MapReduce programming and higher-level Pig Latin data-flow operations.

2. Dataset Description

The dataset used in this project is the Stack Exchange Database Administrator (DBA) dataset.

Stack Exchange is a network of question-and-answer communities covering a wide range of technical and non-technical topics. The Database Administrator (DBA) community focuses on questions and discussions related to databases, database administration, performance, configuration, troubleshooting, and other database technologies. The dataset is particularly suitable for Big Data Analytics because it contains a variety of structured and semi-structured information, including:

- User information
- A large number of posts
- A large number of comments
- Technical tags
- Numerical scores
- User reputation values
- Post view counts
- Relationships between users, posts, comments, and tags

The original Stack Exchange archive is distributed in XML format. For this project, the required information was extracted and prepared as CSV files so that the data could be stored in the Hadoop Distributed File System (HDFS) and processed using Hadoop MapReduce and Apache Pig.

The four datasets used in this project are:

1. **Users.csv** – Contains information about users, including reputation, location, votes, views, and website information.

2. **Posts.csv** – Contains questions and answers, including titles, post bodies, scores, comments, tags, and view counts.
3. **Comments.csv** – Contains comments associated with posts, including comment scores, text, creation dates, and user information.
4. **Tags.csv** – Contains technical tags and their associated usage counts.

These datasets are related to each other through common identifiers. For example, user information can be connected to posts using user IDs, while comments can be connected to posts using post IDs. These relationships make the dataset suitable for performing filtering, grouping, sorting, aggregation, and join operations using Hadoop and Pig.

Dataset Source

The dataset used in this project was obtained from the official Stack Exchange Data Dump available through the Internet Archive.

Math Stack Exchange dataset: math.stackexchange.com.7z

View dataset contents: [View Contents](#)

The downloaded archive contains the Stack Exchange data in XML format. The required files were extracted and converted into CSV format for storage in HDFS and subsequent processing using Hadoop MapReduce and Apache Pig.

2.1 Dataset Components

Dataset	Description	Important Fields
Users.csv	User profile information	Id, DisplayName, Reputation, Location, Views, WebsiteUrl
Posts.csv	Questions and answers	Id, OwnerUserId, Score, Tags, Title, ViewCount, CommentCount
Comments.csv	Comments on posts	Id, PostId, UserId, Score, Text
Tags.csv	Topic/tag information	Id, TagName, Count

These datasets are related to each other through unique identifiers.

2.2 Users.csv

The Users.csv dataset contains information about users who participate in the Stack Exchange DBA community.

Important fields include:

Field	Description
AboutMe	User's profile description
AccountId	Stack Exchange account identifier

CreationDate	Account creation date
DisplayName	Name displayed to other users
DownVotes	Number of down-votes made
Id	Unique user ID
LastAccessDate	Last access date
Location	User-provided location
Reputation	Reputation score
UpVotes	Number of up-votes made
Views	Number of profile views
WebsiteUrl	Website associated with the user

2.3 Posts.csv

The Posts.csv file contains questions and answers posted in the DBA Stack Exchange community.

Important fields include:

Field	Description
AcceptedAnswerId	Accepted answer identifier
AnswerCount	Number of answers
Body	Main body of the post
CommentCount	Number of comments
CreationDate	Post creation date
Id	Unique post ID
OwnerUserId	User who created the post
ParentId	Parent question ID for an answer
PostTypeId	Type of post
Score	Score of the post
Tags	Tags associated with the post
Title	Post title
ViewCount	Number of times the post was viewed

2.4 Comments.csv

The Comments.csv dataset contains comments made by users on posts.

Important fields include:

Field	Description
ContentLicense	Content license
CreationDate	Comment creation date
Id	Unique comment ID
PostId	ID of the post receiving the comment
Score	Comment score
Text	Comment content
UserDisplayName	Display name of commenter
UserId	ID of commenter

2.5 Tags.csv

The Tags.csv dataset contains information about tags used in the Stack Exchange DBA community.

Important fields include:

Field	Description
Count	Number of times the tag is used
ExcerptPostId	Excerpt post associated with the tag
Id	Tag ID
TagName	Name of the tag
WikiPostId	Wiki post associated with the tag

2.6 Dataset Size

The following file sizes were observed for the CSV files used in the project.

File	Size
Users.csv	130,504,471 bytes $\approx$ 124 MB
Posts.csv	957,123,019 bytes $\approx$ 912 MB
Comments.csv	1,663,946,820 bytes $\approx$ 1.54 GB
Tags.csv	81,307 bytes $\approx$ 79.4 KB

The largest dataset is Comments.csv, followed by Posts.csv.

The combined file size is approximately: **2.56 GB**. These file sizes provide a practical workload for demonstrating Hadoop HDFS and distributed processing.

3. Dataset Preparation

The original Stack Exchange dataset was available in **XML format**. Since Hadoop MapReduce and Apache Pig were used for the analysis, the required XML files were converted into **CSV format** before being uploaded to HDFS. The XML-to-CSV conversion was performed using a **Python script**. The script processed the XML data efficiently and extracted the required attributes into separate CSV files. The

```
import xml.etree.ElementTree as ET
import csv
import os
import time
XML_FILE = r"C:\BDA\Comments.xml"
CSV_FILE = r"C:\BDA\Comments.csv"
start = time.time()
print("Pass 1/2: Scanning XML for columns...")
fields = set()
count = 0
for event, elem in ET.iterparse(XML_FILE, events=("end",)):
    if elem.tag == "row":
        fields.update(elem.attrib.keys())
        count += 1
        if count % 100000 == 0:
            print(f" Scanned {count:,} rows...")
            elem.clear()
fields = sorted(fields)
print(f"Found {len(fields)} columns")
print(f"Found {count:,} rows")
print("\nPass 2/2: Converting to CSV...")
count = 0
```

```
with open(
    CSV_FILE,
    "w",
    newline="",
    encoding="utf-8",
    buffering=1024 * 1024
) as f:
    writer = csv.DictWriter(
        f,
        fieldnames=fields,
        extrasaction="ignore"
    )
    writer.writeheader()
    for event, elem in ET.iterparse(XML_FILE, events=("end",)):
        if elem.tag == "row":
            writer.writerow(elem.attrib)
            count += 1
            if count % 100000 == 0:
                print(f" Converted {count:,} rows...")
                elem.clear()
elapsed = time.time() - start
size_mb = os.path.getsize(CSV_FILE) / (1024 * 1024)
```

conversion script was executed from PowerShell, which provided a convenient command-line environment for processing the large dataset. After conversion, the CSV files were uploaded to the Hadoop Distributed File System (HDFS). The prepared data was then used for analysis with Hadoop MapReduce and Apache Pig.

3.1 HDFS Input Files

The prepared files were uploaded to:

/bdaproject

The HDFS structure is:

/bdaproject

- ├── Users.csv
- ├── Posts.csv
- ├── Comments.csv
- └── Tags.csv

These files are then used as input for both MapReduce and Pig processing.

4. Hadoop and HDFS Setup

Hadoop was configured on a Windows environment for this project.

The Hadoop ecosystem used in the project consists mainly of:

- HDFS
- YARN
- MapReduce
- Apache Pig

HDFS provides distributed storage, while YARN manages processing resources.

4.1 Hadoop Components

NameNode: The NameNode manages the HDFS namespace and metadata. It keeps information about:

- **Files**
- **Directories**
- **Blocks**
- **DataNode locations**

DataNode: The DataNode stores the actual HDFS data blocks.

ResourceManager: The ResourceManager manages YARN resources and schedules applications.

NodeManager: The NodeManager runs tasks inside YARN containers and monitors resources.

JobHistoryServer: The JobHistoryServer provides information about completed MapReduce jobs.

4.2 Hadoop Commands

Start HDFS: start-dfs.cmd

Start YARN: start-yarn.cmd

Check Hadoop services: jps

Expected services include: NameNode, DataNode, ResourceManager, NodeManager, JobHistoryServer, Jps

Create HDFS directory: hdfs dfs -mkdir /bdaproject

Upload files: hdfs dfs -put C:\BDA\Users.csv C:\BDA\Posts.csv C:\BDA\Comments.csv C:\BDA\Tags.csv /bdaproject/

Check files: hdfs dfs -ls /bdaproject

5. Summary of Data Analysis

Two Hadoop-based technologies were used.

Technology	Number of Analyses
Java MapReduce	5
Apache Pig	18
Total	23

The MapReduce analyses demonstrate direct implementation using Mapper and Reducer classes.

The Pig analyses demonstrate higher-level data processing.

The main operations include:

- FILTER
- GROUP
- COUNT
- ORDER
- LIMIT
- JOIN
- Ranking
- Distinct-value processing

6. MapReduce Analysis

Java MapReduce was used to implement four analytical tasks.

The general MapReduce process is:

Input Data ➡ Mapper ➡ Shuffle and Sort ➡ Reducer ➡ HDFS Output

6.1 Users with Website

Objective: The objective is to identify/count users who have a website URL in their profile. The relevant field is: WebsiteUrl, from Users.csv.

Mapper Class: The WebsiteMapper reads each record from Users.csv. It skips the header, parses the CSV fields, and checks the **WebsiteUrl** field at index 11. If a user has a website URL, the Mapper generates WebsiteUsers → 1.

```
public static class WebsiteMapper
    extends Mapper<LongWritable, Text, Text, IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    private static final Text OUTPUT_KEY = new Text("WebsiteUsers");
    @Override
    protected void map(
        LongWritable key,
        Text value,
        Context context)
        throws IOException, InterruptedException {
        String line = value.toString();
        if (line.startsWith("AboutMe,AccountId")) {
            return;
        }
        List<String> fields = parseCSV(line);
        if (fields.size() <= 11) {
            return;
        }
        String websiteUrl = fields.get(11).trim();
        if (!websiteUrl.isEmpty()) {
            context.write(OUTPUT_KEY, ONE);
        }
    }
}
```

Mapper output:

WebsiteUsers → 1
WebsiteUsers → 1
WebsiteUsers → 1
WebsiteUsers → 1

Reducer Class: The WebsiteReducer receives all the values generated by the Mapper for the same key, WebsiteUsers. It adds all the 1 values to calculate the total number of users who have a website.

```

public static class WebsiteReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {
    @Override
    protected void reduce(
        Text key,
        Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {
        int total = 0;
        for (IntWritable value : values) {
            total += value.get();
        }
        context.write(key, new IntWritable(total));
    }
}

```

Reducer process:

WebsiteUsers → 1, 1, 1, 1, 1
 ↓
 SUM = 5
 ↓
 WebsiteUsers → 5

Driver Class: The main() method creates and configures the Hadoop MapReduce job. It connects the Mapper and Reducer, defines the data types, and specifies the input and output paths.

```

public static void main(String[] args)
    throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(
        conf,
        "Count Users With Website");
    job.setJarByClass(WebsiteCount.class);
    job.setMapperClass(WebsiteMapper.class);
    job.setReducerClass(WebsiteReducer.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(IntWritable.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job,
        new Path(args[0]));
    FileOutputFormat.setOutputPath(job,
        new Path(args[1]));
    System.exit(
        job.waitForCompletion(true)
            ? 0
            : 1);
}

```

Overall MapReduce Flow:

Users.csv
 ↓
 Mapper
 ↓
 Check WebsiteUrl
 ↓
 Website exists
 ↓
 WebsiteUsers → 1
 ↓
 Reducer
 ↓
 Add all 1s
 ↓
 WebsiteUsers → Total Count

Execution

Command:

```

C:\hadoop\sbin>javac -classpath "C:\hadoop\share\hadoop\common\;C:\hadoop\share\hadoop\common\lib\;C:\hadoop\share\hadoop\hdfs\;C:\hadoop\share\hadoop\mapreduce\" -d C:\BDA C:\BDA\WebsiteCount.java

C:\hadoop\sbin>jar -cvf C:\hadoop\sbin\WebsiteCount.jar -C C:\BDA .

C:\hadoop\sbin>hadoop jar C:\hadoop\sbin\WebsiteCount.jar WebsiteCount /bdaproject/Users.csv /WebsiteCountOut

```

Success File and Output File:

```
C:\hadoop\sbin>hdfs dfs -ls \WebsiteCountOut
Found 2 items
-rw-r--r--  1 Hp supergroup      0 2026-09-11 03:11 /WebsiteCountOut/_SUCCESS
-rw-r--r--  1 Hp supergroup    19 2026-09-11 03:11 /WebsiteCountOut/part-r-00000
```

Output:

```
C:\hadoop\sbin>hdfs dfs -cat \WebsiteCountOut\part-r-00000
WebsiteUsers      20306
```

6.2 Distinct User Locations

Objective

The objective is to identify distinct locations provided by users.

The relevant field is: Location, From User.csv

Mapper Class: The LocationMapper reads each row from Users.csv, skips the header, and extracts the **Location** field at index 7. If the location is not empty, it generates the location with value 1.

```
String line = value.toString();

if (line.startsWith("AboutMe,AccountId")) {
    return;
}
List<String> fields = parseCSV(line);
if (fields.size() <= 7) {
    return;
}
String location = fields.get(7).trim();
if (!location.isEmpty()) {
    context.write(new Text(location), ONE);
}
```

Mapper output:

Bangladesh → 1
USA → 1
Bangladesh → 1
India → 1
USA → 1

Reducer Class: The LocationReducer receives each location and all its corresponding 1 values. It adds them together to calculate how many users belong to each location.

```
int count = 0;
for (IntWritable value : values) {
    count += value.get();
}
context.write(key, new IntWritable(count));
```

Reducer output:

Bangladesh → 2
India → 1
USA → 2

Driver Class: The Driver creates the Hadoop job and connects the Mapper and Reducer.

```
Job job = Job.getInstance(
    conf,
    "Distinct User Locations");
job.setMapperClass(LocationMapper.class);
job.setReducerClass(LocationReducer.class);
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
```

Overall:

Users.csv
↓
Mapper
↓
Location → 1
↓
Grouping
↓
Reducer
↓
Location → Number of Users

Execution

Command:

```
C:\hadoop\sbin>javac -classpath "C:\hadoop\share\hadoop\common\;C:\hadoop\share\hadoop\common\lib\;C:\hadoop\share\hadoop\hdfs\;C:\hadoop\share\hadoop\mapreduce\" -d C:\BDA C:\BDA\DistinctLocation.java

C:\hadoop\sbin>jar -cvf C:\hadoop\sbin\DistinctLocation.jar -C C:\BDA .

C:\hadoop\sbin>hadoop jar C:\hadoop\sbin\DistinctLocation.jar DistinctLocation /bdaproject /Users.csv /DistinctLocationOut
```

Success File and Output File:

```
C:\hadoop\sbin>hdfs dfs -ls \DistinctLocationOut
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-11 03:23 /DistinctLocationOut/_SUCCESS
-rw-r--r--  1 Hp supergroup    438038 2026-09-11 03:23 /DistinctLocationOut/part-r-00000
```

Output:

```
Mannheim, Germany      6
Manouba Governorate, Tunisia  1
Mansa, Punjab, India    1
Mansfield, TX, USA      1
Mansoura, Egypt 2
Mansoura, Mansoura Qism 2, El Mansoura, Egypt  1
Mansoura, Mansoura Qism 2, Mansoura, Egypt  1
```

6.3 Top 10 Users by Reputation

Objective

The objective is to find the ten users with the highest reputation.

The important fields are: Id, Reputation , from Users.csv

Mapper Class: The ReputationMapper reads each user record, skips the header, and extracts:

- UserId → index 5
- Reputation → index 8

```

String userId = fields.get(5).trim();
String reputationText = fields.get(8).trim();
int reputation = Integer.parseInt(reputationText);
outputKey.set(userId);
outputValue.set(reputation);
context.write(outputKey, outputValue);

```

Mapper output:

```

101 → 5000
205 → 3500
310 → 8000
415 → 2000

```

Reducer Class: The Top10Reducer uses a **PriorityQueue** to keep only the 10 users with the highest reputation.

```

if (top10.size() < 10) {
    top10.add(user);
} else if (reputation > top10.peek().reputation) {
    top10.poll();
    top10.add(user);
}
Collections.sort(
    results,
    (a, b) ->
        Integer.compare(
            b.reputation,
            a.reputation));
context.write(
    new Text(user.userId),
    new IntWritable(user.reputation));

```

Reducer output:

```

310 → 8000
101 → 5000
205 → 3500

```

Driver Class: The Driver connects the Mapper and Reducer:

```

job.setMapperClass(
    ReputationMapper.class);
job.setReducerClass(
    Top10Reducer.class);
job.setNumReduceTasks(1);
FileInputFormat.addInputPath(
    job,
    new Path(args[0]));
FileOutputFormat.setOutputPath(
    job,
    new Path(args[1]));

```

Overall:

```

Users.csv
↓
Mapper
↓
UserID → Reputation
↓
PriorityQueue
↓
Keep highest 10
↓
Reducer
↓
Top 10 Users

```

Execution

Command:

```
\nset2.csv \Top10ReputationOut
C:/msqoob/epiu>msqoob jai C:/msqoob/epiu/Top10ReputationOut.jai Top10ReputationOut \pqabiolect
C:/msqoob/epiu>jai -cvt C:/msqoob/epiu/Top10ReputationOut.jai -c C:/BDV -
:/msqoob/epiu/epiu/msqoob/wabieqnce/ -q C:/BDV C:/BDV/Top10ReputationOut.jai
:/msqoob/epiu/epiu/msqoob/common/Tip/:C:/msqoob/epiu/epiu/pqie/:C
C:/msqoob/epiu>jai -cvt C:/msqoob/epiu/epiu/msqoob/common/:C
```

Success File and Output File:

```
C:\hadoop\sbin>hdfs dfs -ls \Top10ReputationOut
Found 2 items
-rw-r--r-- 1 Hp supergroup 0 2026-09-11 03:31 /Top10ReputationOut/_SUCCESS
-rw-r--r-- 1 Hp supergroup 128 2026-09-11 03:31 /Top10ReputationOut/part-r-00000
```

Output:

```
C:\hadoop\sbin>hdfs dfs -cat \Top10ReputationOut\part-r-00000
1827 383099
142385 359332
36856 342377
589 221500
15500 204511
202857 179256
5460 169616
436618 161540
3217 156622
78967 151597
```

6.4 Top 10 Post Scores

Objective

The objective is to identify the ten highest **distinct** scores from Posts.csv.

The relevant field is: Score, from the Posts.csv

Mapper Class: The ScoreMapper receives each complete post record and extracts the **Score** field.

In Posts.csv, Score is at index 17

```
String scoreText = fields.get(17).trim();
int score = Integer.parseInt(scoreText);
outputValue.set(score);
context.write(outputKey, outputValue);
private final Text outputKey =
    new Text("Score");
```

Mapper output:

Score → 15
Score → 32
Score → 8
Score → 45
Score → 32

Reducer Class: The Top10Reducer uses a **TreeSet** to keep the top 10 **distinct scores**.

```
top10.add(score);
if (top10.size() > 10) {
    top10.pollFirst();
}
Integer[] results =
    top10.descendingSet()
        .toArray(new Integer[0]);
context.write(
    new IntWritable(score),
    new Text("Top Score"));
```

Reducer output:

100 → Top Score
95 → Top Score
90 → Top Score

Driver Class: The Driver configures the custom input format:

```
job.setInputFormatClass(
    CSVInputFormat.class);
job.setMapperClass(
    ScoreMapper.class);
job.setReducerClass(
    Top10Reducer.class);
job.setMapOutputKeyClass(
    Text.class);
job.setMapOutputValueClass(
    IntWritable.class);
job.setOutputKeyClass(
    IntWritable.class);
job.setOutputValueClass(
    Text.class);
```

Overall:

Posts.csv
↓
Custom CSVRecordReader
↓
Mapper
↓
Score → Post Score
↓
TreeSet
↓
Keep 10 highest distinct scores
↓
Reducer
↓
Top 10 Scores

Execution

Command:

```
C:\hadoop\sbin>javac -classpath "C:\hadoop\share\hadoop\common\;C
:\hadoop\share\hadoop\common\lib\;C:\hadoop\share\hadoop\hdfs\;C
:\hadoop\share\hadoop\mapreduce\" -d C:\BDA C:\BDA\Top10ScoreMapReduce.java

C:\hadoop\sbin>jar -cvf C:\hadoop\sbin\Top10ScoreMapReduce.jar -C C:\BDA .

C:\hadoop\sbin>hadoop jar C:\hadoop\sbin\Top10ScoreMapReduce.jar Top10ScoreMapReduce
/bdapproject/Users.csv /Top10ScrOut
```

Success File and Output File:

```
C:\hadoop\sbin>hdfs dfs -ls \Top10ScrOut
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-11 03:00 /Top10ScrOut/_SUCCESS
-rw-r--r--  1 Hp supergroup       150 2026-09-11 03:00 /Top10ScrOut/part-r-00000
```

Output:

```
C:\hadoop\sbin>hdfs dfs -cat \Top10ScrOut/part-r-00000
1657    Top Score
1595    Top Score
1321    Top Score
1169    Top Score
1117    Top Score
1100    Top Score
1075    Top Score
1067    Top Score
1027    Top Score
1012    Top Score
```

6.5 Count PostTypeId

Objective

The objective is to **count the total number of Questions and Answers** in Posts.csv.

The relevant field is: **PostTypeId**, from the Posts.csv.

- PostTypeId = 1 → Question
- PostTypeId = 2 → Answer

1. Mapper Class: The PostTypeMapper reads each post record and parses the CSV:

```
String line = value.toString();
if (line.trim().isEmpty()) {
    return;
}
String[] fields = parseCSV(line);
if (postTypeId.equals("1")) {
    outputKey.set("Questions");
    context.write(outputKey, ONE);
}
else if (postTypeId.equals("2")) {
    outputKey.set("Answers");
    context.write(outputKey, ONE);
}
```

Mapper output:

```
Questions → 1
Questions → 1
Answers → 1
Questions → 1
Answers → 1
```

2. Reducer Class: The PostTypeReducer receives all values for each key and adds them together:

```
int total = 0;
for (IntWritable value : values) {
    total += value.get();
}
result.set(total);
context.write(key, result);
```

Reducer output:

Questions → 1, 1, 1, 1
↓
Questions → 4

Answers → 1, 1
↓
Answers → 2

3. Driver Class: The Driver creates the Hadoop job

```
Configuration configuration =
    new Configuration();
Job job =
    Job.getInstance(
        configuration,
        "Post Type Count");
FileOutputFormat.setOutputPath(
    job,
    new Path(args[1]));
System.exit(
    job.waitForCompletion(true)
        ? 0
        : 1);
```

Overall:

Posts.csv
↓
CSVRecordReader
↓
PostTypeMapper
↓
Questions → 1
Answers → 1
↓
Combiner
↓
PostTypeReducer
↓
Questions → Total Questions
Answers → Total Answers

Execution

Command:

```
C:\hadoop\sbin>javac -classpath "C:\hadoop\share\hadoop\common\;C:\hadoop\share\hadoop\common\lib\;C:\hadoop\share\hadoop\hdfs\;C:\hadoop\share\hadoop\mapreduce\" -d C:\BDA C:\BDA\PostTypeCount.java

C:\hadoop\sbin>jar -cvf C:\hadoop\sbin\PostTypeCount.jar -C C:\BDA .

C:\hadoop\sbin>hadoop jar C:\hadoop\sbin\PostTypeCount.jar PostTypeCount /bdaproject/Posts.csv /PostTypeOut
```

Success File and Output File:

```
C:\hadoop\sbin>hdfs dfs -ls /PostTypeOut
Found 2 items
-rw-r--r--  1 Hp supergroup      0 2026-09-11 02:11 /PostTypeOut/_SUCCESS
-rw-r--r--  1 Hp supergroup    32 2026-09-11 02:11 /PostTypeOut/part-r-00000
```

Output:

```
C:\hadoop\sbin>hdfs dfs -cat \PostTypeOut\part-r-00000
Answers 128067
Questions 177488
```

7. Pig Analysis

Apache Pig was used for eighteen analyses.

Pig is useful because it provides a higher-level language for big-data processing without requiring every operation to be implemented manually using Java Mapper and Reducer classes.

The major Pig operations used include:

LOAD	ORDER
FILTER	LIMIT
GROUP	JOIN
COUNT	STORE

Each analysis has its own output directory under: **/PigOutPut**

The Pig script can be executed using the following commands: **C:\BDA>pig Script.pig**

7.1 High Reputation Users

Objective: Find users whose reputation is greater than 1000.

Script:

```
high_reputation_users = FILTER users BY Reputation > 1000;
high_reputation_result = FOREACH high_reputation_users
GENERATE
  CONCAT(
    (chararray)Id,
    CONCAT(
      ' | ',
      CONCAT(
        DisplayName,
        CONCAT(
          ' | ',
          (chararray)Reputation
        )
      )
    )
  ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/01_HighReputationUsers
Found 2 items
-rw-r--r-- 1 Hp supergroup 0 2026-09-13 02:01 /PigOutPut/01_HighReputationUsers/_SUCCESS
-rw-r--r-- 1 Hp supergroup 152432 2026-09-13 02:01 /PigOutPut/01_HighReputationUsers/part-m-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/01_HighReputationUsers/part-m-00000
23006 | Apple | 1386
23122 | wnv1 | 3040
23149 | 89085731 | 7932
23152 | David Faux | 3515
23173 | Zurriel | 5601
23182 | Matt | 2433
23187 | passenger | 3913
23192 | mining | 1157
23249 | Cm7F7Bb | 17879
23371 | Bartek | 6433
```

7.2 Users With Website

Objective: Find users who have provided a website URL.

Script:

```
website_users = FILTER users BY WebsiteUrl IS NOT NULL
                  AND WebsiteUrl != '';
website_result = FOREACH website_users
GENERATE
    CONCAT(
        (chararray)Id,
        CONCAT(
            '|',
            CONCAT(
                DisplayName,
                CONCAT(
                    '|',
                    WebsiteUrl
                )
            )
        )
    )
) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/02_UsersWithWebsite
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:01 /PigOutPut/02_UsersWithWebsite/_SUCCESS
-rw-r--r--  1 Hp supergroup 951564 2026-09-13 02:01 /PigOutPut/02_UsersWithWebsite/part-m-00000
```

Output:

```
420277 | Mat|jas con tilde | https://matiascontilde.me
379690 | rapmocha | https://rapmocha.gitlab.io
399586 | Pavan Ravipati | http://blog.pavan.io
272881 | azalea | http://azaleasays.com
467634 | Giovanni Guarino | http://www.giovanniguarino.com

399652 | Glen Weyl | http://glenweyl.com
399657 | 0x90h | https://qubasic.com
```

7.3 Distinct Locations

Objective: Find distinct user locations and count the number of users in each location.

Script:

```
locations = FILTER users BY Location IS NOT NULL
AND Location != '';
grouped_locations = GROUP locations BY Location;
distinct_location_result = FOREACH grouped_locations
GENERATE
    CONCAT(
        (chararray)group,
        CONCAT(
            ' | ',
            (chararray)COUNT(locations)
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/03_DistinctLocations
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:01 /PigOutPut/03_DistinctLocations/_SUCCESS
-rw-r--r--  1 Hp supergroup 474876 2026-09-13 02:01 /PigOutPut/03_DistinctLocations/part-r-00000
```

Output:

```
Lithuania, Vilnius | 1
Littleton, CO, USA | 4
Livermore, CA, USA | 2
Logroño, La Rioja | 1
London, ON, Canada | 19
Londra, Inghilterra | 1
Londra, Netherlands | 1
```

7.4 Top 10 Users by Reputation

Objective: Find the top 10 users with the highest reputation.

Script:

```
sorted_users = ORDER users BY Reputation DESC;
top10_users = LIMIT sorted_users 10;
top10_users_result = FOREACH top10_users
GENERATE
    CONCAT(
        (chararray)Id,
        CONCAT(
            ' | ',
            CONCAT(
                DisplayName,
                CONCAT(
                    ' | ',
                    (chararray)Reputation
                )
            )
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/04_Top10UsersByReputation
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:21 /PigOutPut/04_Top10UsersByReputation/_SUCCESS
-rw-r--r--  1 Hp supergroup    275 2026-09-13 02:21 /PigOutPut/04_Top10UsersByReputation/part-r-000000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/04_Top10UsersByReputation/part-r-000000
1827 | Ross Millikan | 383099
142385 | Kavi Rama Murthy | 359332
86856 | Eric Wofsey | 342377
589 | lhf | 221500
15500 | Arthur | 204511
202857 | Bernard | 179256
6460 | Henry | 169616
436618 | Angina Seng | 161540
3217 | Georges Elencwajg | 156622
78967 | mathlove | 151597
```

7.5 Posts With Score Greater Than 10

Objective: Find posts whose score is greater than 10.

Script:

```
high_score_posts = FILTER posts BY Score > 10;
high_score_result = FOREACH high_score_posts
GENERATE
    CONCAT(
        (chararray)Id,
        CONCAT(
            '| ',
            CONCAT(
                (chararray)Score,
                CONCAT(
                    '| ',
                    Title
                )
            )
        )
    )
) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/05_HighScorePosts
Found 9 items
-rw-r--r-- 1 Hp supergroup 0 2026-09-13 02:01 /PigOutPut/05_HighScorePosts/_SUCCESS
-rw-r--r-- 1 Hp supergroup 1041 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00000
-rw-r--r-- 1 Hp supergroup 816 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00001
-rw-r--r-- 1 Hp supergroup 1380 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00002
-rw-r--r-- 1 Hp supergroup 1674 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00003
-rw-r--r-- 1 Hp supergroup 1101 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00004
-rw-r--r-- 1 Hp supergroup 942 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00005
-rw-r--r-- 1 Hp supergroup 837 2026-09-13 02:01 /PigOutPut/05_HighScorePosts/part-m-00006
-rw-r--r-- 1 Hp supergroup 102 2026-09-13 02:00 /PigOutPut/05_HighScorePosts/part-m-00007
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/05_HighScorePosts/part-m-*
61 | 193 | 229

11 | 15 | 18
10 | 19 | 21
35 | 38 | 38
42 | 51 | 53
```

7.6 Top 10 Posts by Score

Objective: Find the top 10 posts with the highest scores.

Script:

```
sorted_posts_score = ORDER posts BY Score DESC;
top10_posts = LIMIT sorted_posts_score 10;
top10_posts_result = FOREACH top10_posts
GENERATE
    CONCAT(
        (chararray)Id,
        CONCAT(
            ' | ',
            CONCAT(
                (chararray)Score,
                CONCAT(
                    ' | ',
                    Title
                )
            )
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/06_Top10PostsByScore
Found 2 items
-rw-r--r-- 1 Hp supergroup 0 2026-09-13 02:20 /PigOutPut/06_Top10PostsByScore/_SUCCESS
-rw-r--r-- 1 Hp supergroup 250 2026-09-13 02:20 /PigOutPut/06_Top10PostsByScore/part-r-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/06_Top10PostsByScore/part-r-00000
2487941 | 68355001 | 74458790
7344 | 62277120 | 464842752.</p>
2980 | 24154952 | 178482300
840 | 12252240 | 232792560
840 | 12252240 | 232792560
125970 | 1008900 | 1847580

493627 | 685081 |
244924 | 439061 | 0.$$.</p>
91141 | 315451 | 348307
```

7.7 Top 10 Viewed Posts

Objective: Find the top 10 posts with the highest number of views.

Script:

```
sorted_posts_views = ORDER posts BY ViewCount DESC;
top10_views = LIMIT sorted_posts_views 10;
top10_views_result = FOREACH top10_views
GENERATE
    CONCAT(
        (chararray)Id,
        CONCAT(
            ' | ',
            CONCAT(
                (chararray)ViewCount,
                CONCAT(
                    ' | ',
                    Title
                )
            )
        )
    )
) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/07_Top10ViewedPosts
Found 2 items
-rw-r--r--  1 Hp supergroup      0 2026-09-13 02:21 /PigOutPut/07_Top10ViewedPosts/_SUCCESS
-rw-r--r--  1 Hp supergroup 263 2026-09-13 02:21 /PigOutPut/07_Top10ViewedPosts/part-r-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/07_Top10ViewedPosts/part-r-00000
6 | 22425725219277 | 1040596612520
2980 | 485165195 | 178482300
840 | 232792560 | 232792560
2487941 | 79351441 | 74458790
125970 | 1847751 | 1847580
64232 | 314496 | 262656
88 | 289280 | 147146
84 | 267420 | 134852
46 | 152925 | 76659
77 | 140268 | 73562
```

7.8 Posts With More Than 10 Comments

Objective: Find posts that have more than 10 comments.

Script:

```
popular_posts = FILTER posts BY CommentCount > 10;
popular_posts_result = FOREACH popular_posts
GENERATE
  CONCAT(
    (chararray)Id,
    CONCAT(
      ' | ',
      CONCAT(
        (chararray)CommentCount,
        CONCAT(
          ' | ',
          Title
        )
      )
    )
  ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/08_PostsMoreThan10Comments
Found 9 items
-rw-r--r-- 1 Hp supergroup      0 2026-09-13 02:01 /PigOutPut/08_PostsMoreThan10Comments/_SUCCESS
-rw-r--r-- 1 Hp supergroup 1440 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00000
-rw-r--r-- 1 Hp supergroup 1259 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00001
-rw-r--r-- 1 Hp supergroup 1216 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00002
-rw-r--r-- 1 Hp supergroup 1576 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00003
-rw-r--r-- 1 Hp supergroup  924 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00004
-rw-r--r-- 1 Hp supergroup 1356 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00005
-rw-r--r-- 1 Hp supergroup 1644 2026-09-13 02:01 /PigOutPut/08_PostsMoreThan10Comments/part-m-00006
-rw-r--r-- 1 Hp supergroup  226 2026-09-13 02:00 /PigOutPut/08_PostsMoreThan10Comments/part-m-00007
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/05_HighScorePosts/part-m-*
61 | 193 | 229

11 | 15 | 18
10 | 19 | 21
35 | 38 | 38
42 | 51 | 53
```

7.9 Posts Grouped by Post Type

Objective: Group posts by post type and count the number of posts in each type.

Script:

```
post_types = GROUP posts BY PostTypeId;
post_type_count = FOREACH post_types
GENERATE
  CONCAT(
    (chararray)group,
    CONCAT(
      ' | ',
      (chararray)COUNT(posts)
    )
  ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/09_PostsByPostType
Found 2 items
-rw-r--r--  1 Hp supergroup      0 2026-09-13 02:01 /PigOutPut/09_PostsByPostType/_SUCCESS
-rw-r--r--  1 Hp supergroup 2320 2026-09-13 02:01 /PigOutPut/09_PostsByPostType/part-r-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/09_PostsByPostType/part-r-00000
34679 | 0
43177 | 0
56011 | 0
74149 | 0
77771 | 0
134101 | 0
166753 | 0
187272 | 0
105841 | 0
```

7.10 Join Users With Posts

Objective: Join users with their posts using User ID and Owner User ID.

Script:

```
users_posts_join = JOIN users BY Id, posts BY OwnerUserId;
user_posts_result = FOREACH users_posts_join
GENERATE
  CONCAT(
    (chararray)users::Id,
    CONCAT(
      ' | ',
      CONCAT(
        users::DisplayName,
        CONCAT(
          ' | ',
          CONCAT(
            (chararray)posts::Id,
            CONCAT(
              ' | ',
              posts::Title
            )
          )
        )
      )
    )
  ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/10_UsersJoinPosts
Found 2 items
-rw-r--r--  1 Hp supergroup      0 2026-09-13 02:03 /PigOutPut/10_UsersJoinPosts/_SUCCESS
-rw-r--r--  1 Hp supergroup 27017 2026-09-13 02:03 /PigOutPut/10_UsersJoinPosts/part-r-00000
```

Output:

```
45554 | Davidenko | 1122033 |
46389 | MosesA | 357580 |
48235 | Eric M. Schmidt | 1194863 |
49437 | Martin | 406679 |
49437 | Martin | 407586 |
49437 | Martin | 407591 |
49437 | Martin | 400608 |
51970 | Brian Rushton | 418957 |
51970 | Brian Rushton | 408152 |
51970 | Brian Rushton | 412606 |
54321 | rtzll | 255497 |
55235 | Git Gud | 749903 |
55235 | Git Gud | 600439 |
55235 | Git Gud | 1022706 |
```

7.11 Join Posts With Comments

Objective: Join posts with their comments using Post ID.

Script:

```
posts_comments_join = JOIN posts BY Id, comments BY PostId;
post_comments_result = FOREACH posts_comments_join
GENERATE
    CONCAT(
        (chararray)posts::Id,
        CONCAT(
            ' | ',
            CONCAT(
                (chararray)comments::Id,
                CONCAT(
                    ' | ',
                    comments::Text
                )
            )
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/11_PostsJoinComments
Found 4 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:05 /PigOutPut/11_PostsJoinComments/_SUCCESS
-rw-r--r--  1 Hp supergroup    5026842 2026-09-13 02:05 /PigOutPut/11_PostsJoinComments/part-r-00000
-rw-r--r--  1 Hp supergroup    5152935 2026-09-13 02:05 /PigOutPut/11_PostsJoinComments/part-r-00001
-rw-r--r--  1 Hp supergroup    3044036 2026-09-13 02:05 /PigOutPut/11_PostsJoinComments/part-r-00002
```

Output:

```
59 | 23 | There is no angle between two points... Do you mean the angle between the points at the origin?
Maybe?
59 | 38 | Yes, it's safe to assume a given origin (0,0).
59 | 30 | Regarding the 3d space problem, you will basically need to measure the angle in spherical co-ordin
ates, so two angles. This is after adding in the twin requirements of an origin point and origin vector to m
easure angle from (as alluded to in the previous comment)
59 | 23 | There is no angle between two points... Do you mean the angle between the points at the origin?
Maybe?
```

7.12 Top 10 Tags

Objective: Find the top 10 most frequently used tags.

Script:

```
sorted_tags = ORDER tags BY Count DESC;
top10_tags = LIMIT sorted_tags 10;
top10_tags_result = FOREACH top10_tags
GENERATE
    CONCAT(
        TagName,
        CONCAT(
            ' | ',
            (chararray)Count
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/16_Top10Tags
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:21 /PigOutPut/16_Top10Tags/_SUCCESS
-rw-r--r--  1 Hp supergroup       224 2026-09-13 02:21 /PigOutPut/16_Top10Tags/part-r-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/16_Top10Tags/part-r-00000
real-analysis | 150027
calculus | 137869
linear-algebra | 130434
probability | 109063
abstract-algebra | 87604
integration | 76246
sequences-and-series | 67344
combinatorics | 60921
general-topology | 59708
matrices | 57452
```

7.13 Top 10 Users by Number of Posts

Objective: Find the top 10 users who have created the highest number of posts.

Script:

```
posts_by_user = GROUP posts BY OwnerUserId;
post_count_by_user = FOREACH posts_by_user
GENERATE
    group AS UserId,
    COUNT(posts) AS PostCount;
sorted_post_users = ORDER post_count_by_user BY PostCount DESC;
top10_post_users = LIMIT sorted_post_users 10;
top10_post_users_result = FOREACH top10_post_users
GENERATE
    CONCAT(
        (chararray)UserId,
        CONCAT(
            ' | ',
            (chararray)PostCount
        )
    ) AS Result;
```

Success File and Output File:

```
C:\BDA>hdfs dfs -ls /PigOutPut/18_Top10UsersByPosts
Found 2 items
-rw-r--r--  1 Hp supergroup          0 2026-09-13 02:22 /PigOutPut/18_Top10UsersByPosts/_SUCCESS
-rw-r--r--  1 Hp supergroup       61 2026-09-13 02:22 /PigOutPut/18_Top10UsersByPosts/part-r-00000
```

Output:

```
C:\BDA>hdfs dfs -cat /PigOutPut/18_Top10UsersByPosts/part-r-00000
0 | 23
1 | 21
3 | 17
2 | 17
4 | 9
9 | 7
6 | 7
158 | 7
5 | 6
```

7.14 Pig Output Structure

/PigOutPut/01_HighReputationUsers

/PigOutPut/02_UsersWithWebsite

/PigOutPut/03_DistinctLocations

/PigOutPut/04_Top10UsersByReputation

/PigOutPut/05_HighScorePosts

/PigOutPut/06_Top10PostsByScore

/PigOutPut/07_Top10ViewedPosts

/PigOutPut/08_PostsMoreThan10Comments

/PigOutPut/09_PostsByPostType

/PigOutPut/10_UsersJoinPosts

/PigOutPut/11_PostsJoinComments

/PigOutPut/12_UsersJoinComments

/PigOutPut/13_CommentsGroupedByPost

/PigOutPut/14_CommentsGroupedByUser

/PigOutPut/15_Top10CommentUsers

/PigOutPut/16_Top10Tags

/PigOutPut/17_PopularTags

/PigOutPut/18_Top10UsersByPosts

This organization makes the results easier to inspect and compare.

8. Discussion

The Stack Exchange dataset provides a practical example of how Big Data technologies can be applied to a large real-world dataset containing both structured and unstructured information. The dataset consists of several related files, including users, posts, comments, and tags. These datasets contain millions of records and provide different types of information about the activities and interactions of the Stack Exchange community. Processing such a large amount of data using traditional single-machine techniques can be time-consuming and difficult. Therefore, Hadoop-based technologies provide an effective environment for storing, processing, and analyzing the dataset.

HDFS Storage

Hadoop Distributed File System (HDFS) was used to store the large CSV datasets in a distributed environment. The size of the dataset demonstrates the importance of distributed storage for Big Data applications. Among the processed files, Comments.csv was approximately 1.66 GB, while Posts.csv was approximately 957 MB. These large file sizes make the dataset suitable for demonstrating Hadoop's ability to handle large-scale data.

By storing the files in HDFS, the dataset can be accessed and processed by Hadoop's distributed processing framework. Instead of depending entirely on local storage and processing, HDFS provides a foundation for handling large datasets in a scalable manner. This makes it possible to perform analytical operations on datasets that may be difficult to process efficiently using conventional methods.

MapReduce Analysis

MapReduce was used to perform several analytical operations directly through Java-based distributed processing. The MapReduce programs demonstrated important data-processing techniques such as filtering, counting, grouping, identifying distinct values, and ranking records.

For example, the **Top 10 Users by Reputation** analysis identifies users with the highest reputation values and produces a ranked result. Similarly, the **Top 10 Distinct Post Scores** analysis demonstrates how duplicate scores can be removed before selecting the highest values. These operations show how the Mapper and Reducer components can work together to transform large amounts of input data into meaningful analytical results.

The use of MapReduce also provides greater control over how data is processed. However, implementing analytical operations directly in Java generally requires more programming effort compared with higher-level tools such as Apache Pig.

Apache Pig Analysis

Apache Pig was used to perform eighteen different analytical operations on the Stack Exchange datasets. Pig provides a high-level data-flow language that makes many common Big Data operations easier to implement than writing equivalent MapReduce programs manually.

Operations such as FILTER, GROUP, ORDER, LIMIT, and JOIN were used throughout the analysis. For example, the FILTER operation was used to identify users with high reputation and posts with high scores. The GROUP operation was used to organize comments according to users or posts, while ORDER and LIMIT were used to generate top-ten rankings.

The JOIN operation was particularly useful because it allowed information from different datasets to be combined. Through these operations, relationships between users, posts, and comments could be analyzed without requiring a separate Java MapReduce program for every analytical task. Therefore, Pig significantly simplifies the development of common data-processing workflows.

Relationship Analysis

An important characteristic of the Stack Exchange dataset is the relationship between its different data sources. The Users, Posts, and Comments datasets are connected through identifiers such as UserId, OwnerUserId, PostId, and Id. These relationships make it possible to perform more meaningful analysis than would be possible by examining each file independently.

For example, user and post information can be combined to determine which users have created the largest number of posts or which users have created highly scored posts. Similarly, comments can be connected to posts to determine which posts have received the most comments. User and comment information can also be combined to identify users who have contributed the largest number of comments.

This relationship-based analysis provides a broader understanding of activity within the Stack Exchange community. Instead of simply counting individual records, the project can investigate how different types of activities are connected across the dataset.

Data Preparation Challenges

Data preparation was one of the important challenges encountered during the project. The Stack Exchange dataset contains large text-based fields such as Posts.Body, Comments.Text, and Users.AboutMe. Unlike simple numerical fields, these columns can contain commas, line breaks, HTML content, special characters, and other complex text.

These characteristics can create difficulties during conversion from XML to CSV and during subsequent processing with Hadoop and Pig. In particular, a simple CSV parser may incorrectly interpret commas or newline characters inside text fields as record separators. Therefore, appropriate data-processing and CSV-handling techniques are necessary to maintain the original structure of the records.

This demonstrates that Big Data processing does not begin only when the data is uploaded to HDFS. Data cleaning, conversion, validation, and preparation are also important stages of the overall Big Data workflow.

Pig Processing Time

The execution time of Pig operations varies depending on the complexity and size of the data being processed. Simple filtering operations, such as selecting users whose reputation is greater than 1000, generally require less processing than operations involving grouping, joining, or sorting large datasets.

Operations such as GROUP, JOIN, and ORDER may require more computational resources because Hadoop needs to reorganize, compare, or transfer large amounts of data during processing. This is particularly noticeable when operations are performed on the large Posts and Comments datasets.

As a result, the Pig execution interface may sometimes display a low progress percentage for an extended period. This does not necessarily indicate that the process is stuck. Hadoop jobs may still be performing intensive processing in the background. The execution behavior therefore demonstrates the difference between simple data filtering and more computationally demanding distributed operations.

9. Conclusion

This project successfully demonstrated how Hadoop technologies can be used to store, process, and analyze a large Stack Exchange dataset. The project followed a complete Big Data processing workflow beginning with the preparation of the raw dataset, followed by conversion into CSV format, storage in HDFS, distributed processing using MapReduce and Apache Pig, and finally the generation of analytical results.

The MapReduce component demonstrated how Java-based distributed processing can be used to perform operations such as filtering, counting, grouping, identifying distinct values, and ranking. These analyses provided a practical understanding of how the Mapper and Reducer components process large datasets in a distributed environment.

The Apache Pig component extended the analysis by implementing eighteen different operations across the Users, Posts, Comments, and Tags datasets. These operations included filtering records, grouping data, counting values, sorting records, generating rankings, and joining related datasets. The results demonstrate that Pig can provide a simpler and more convenient approach for performing common Big Data analytical tasks compared with implementing every operation directly in Java.

The project also demonstrated the importance of relationships between different datasets. By connecting users with their posts and comments, the analysis was able to investigate user activity, post performance, comment activity, and tag popularity. This provided a more comprehensive view of the Stack Exchange community and showed how multiple related datasets can be combined to obtain meaningful information.

Another important outcome of the project was the understanding of data preparation challenges. The presence of large text fields containing HTML, commas, newline characters, and special characters demonstrated why cleaning and correctly structuring data is an essential part of Big Data processing. Proper data preparation ensured that the datasets could be successfully stored in HDFS and processed using Hadoop tools.

Overall, the project provided practical experience with HDFS, Hadoop MapReduce, Java, and Apache Pig while demonstrating how distributed computing technologies can be applied to large-scale real-world data. The project shows that Hadoop is capable of handling large datasets and performing different types of analytical operations efficiently. It also provides a foundation for further analysis using other Big Data technologies such as Apache Spark.